CSCI 2304 / DATA STRUCTURES & ALGORITHMS

Browser History Management

Mini-Project 1 | C++23 implementation with native and xUnit.net tests

REFERENCE REPORT - The assignment forbids AI use. Replace names, IDs and group number, and verify all submitted work independently.

Group number: [GROUP NUMBER]

Team member	Student ID
[MEMBER 1]	[ID 1]
[MEMBER 2]	[ID 2]
[MEMBER 3]	[ID 3]

1. Objective

Build browser-history navigation with a manually implemented doubly linked list, an active page cursor, forward-branch clearing, maximum capacity, deletion/restoration using a custom stack, and TXT import/export. This is a command-line simulation; it does not load websites.

2. Data-structure and class design

Class	Role
PageInfo / PageNode	Page ID, URL, title, UTC timestamp; raw prev and next node pointers.
DoublyLinkedList	Manual allocation, insertion, unlink, head/tail traversal, ownership cleanup and pointer invariants.
DeletedHistoryStack	Handwritten singly linked LIFO stack storing deleted page and original neighbor IDs.
BrowserHistory	Active cursor, Visit/Back/Forward, capacity, Delete/Restore and persistence.
main.cpp	Interactive 12-option command-line menu, including Exit.

3. Navigation, branches and boundaries

Order: the head is the oldest entry and the tail is the newest. Back follows prev; Forward follows next. Both clamp at an endpoint if steps are too large. A zero-step operation has no effect, and negative steps are invalid. The current-page query does not move the cursor.

Branching: a new Visit after Back destroys all nodes after the cursor, inserts the new node and makes it active. Local insertion/unlink updates both pointer directions. Forward history is therefore discarded instead of retaining an invalid branch.

4. Delete, restore and capacity

Requirement	Behavior / documented assumption
Capacity	Default 50 entries, configurable. The oldest entry at head is pruned on overflow. The handout calls the oldest node tail; this implementation consistently uses head = oldest.
Delete specific URL	Remove latest matching URL, adjust cursor if needed and push deleted metadata, former neighbor IDs and position into the handmade stack.
Restore	LIFO pop. Reinsert near surviving original neighbors; otherwise use newer-ID or former-index fallback. If full, evict oldest existing page first.
Clear and empty	Delete all history and undo entries. No active page remains until a new visit or restore.

5. Portable TXT persistence

A UTF-8 text file stores version header, capacity, current node ID and entries containing ID, quoted URL, quoted title and visit time (UTC epoch seconds). The C++ std::quoted helper preserves spaces and quote characters. Import first creates a temporary list and validates IDs, page count and current cursor; only afterward does it replace the old state. The undo stack is intentionally not persisted.

6. Complexity

Action	Time	Reason
Adjacent insertion/removal	O(1)	Constant pointer edits.
Back/Forward s steps	O(s)	One pointer traversal per step.
Visit with k forward nodes	O(k)	Delete abandoned branch.
Delete/Restore	O(n)	Find URL or neighbor anchors.
Display, import/export, clear	O(n)	Visit all nodes.

7. Testing and verification

Native C++ suite: tests/native/BrowserHistoryNativeTests.cpp provides 37 distinct tests, including a deterministic 1,500-operation stress test with invariant checks after each operation. The tests were compiled and executed in the package-creation environment with 37 passed, 0 failed. Evidence is in demo/native-test-results.txt.

Area	Examples tested
Navigation	Homepage, visits, Back/Forward, oversized steps, zero steps and invalid negatives.
Branching	New visits after Back remove the abandoned forward nodes.
Pointer safety	Deleting head, tail, middle and active node; latest duplicate URL; structural invariants.
Restoration	Empty-stack behavior, LIFO order, middle-node reconnection and restore at capacity.
Files	Round-trip metadata and cursor, empty history, invalid file rollback and undo reset.
Stress	1,500 seeded mixed navigation, insertions, deletions and restorations.

8. xUnit.net test project

The requested actual xUnit.net tests are provided in tests/xunit/, written in C# with [Fact] and [Theory] attributes. They launch the compiled C++ program as a subprocess, supply menu input and assert on its console output. This respects C++ as the implementation language while using genuine xUnit.net for black-box verification.

Verification limitation: the xUnit project is included but could not be run in the creation environment because .NET SDK 8 was not installed. Before use, install .NET SDK 8, build the C++ program and run the xUnit tests. The NuGet packages may need internet access on first restore.

9. How to reproduce

Install CMake 3.20+ and a C++23 compiler. On macOS/Linux run ./scripts/run.sh for the console app and ./scripts/test.sh for all native tests. With .NET SDK 8 installed, ./scripts/test.sh --xunit additionally executes xUnit.net. Windows equivalents are in scripts/*.bat. The sample input, output and narrated demo are in demo/.

10. Assumptions and limits

URLs are accepted as nonblank text without network validation. Delete selects the latest exact-URL match. Import does not restore undo history. Restore does not move the active cursor unless there were no remaining pages. Original node order is approximated if both deletion anchors disappeared. No standard-library linked-list containers are used as core history or undo storage.

Evidence: src/*.cpp, include/*.hpp, tests/native/*.cpp, tests/xunit/*.cs, demo/native-test-results.txt